NFP 107 – Systèmes de gestion de bases de données - Durée 2h30
Examen première session 15 juin 2022 (7 pages)

Consignes :
— Examen sur table – Seuls documents autorisés : 10 pages maximum de notes manuscrites.

On met en place un système d'information sur les structures administratives françaises : régions, départements, préfectures

Voici le schéma de la base sur laquelle nous allons travailler. Les clés primaires sont en gras, les clés étrangères ne sont pas indiquées.

— Personne(**idPersonne**, nom, prénom)
— Région(**codeRégion**, intitulé, préfecture, idPprésident)
— Département (**codeDpt**, nom, préfecture, population, codeRégion, idPprésident)
— Voisins (**codeDpt1**, **codeDpt2**)

Dans les tables `Région` et `Département`, `préfecture` désigne la ville siège de la préfecture de région ou de département. Strasbourg est à la fois préfecture de la région *Grand Est* et du département du Bas-Rhin. Dans la même région, Colmar est préfecture du département du Haut-Rhin. Régions et départements sont présidés par une personne.

Deux départements sont voisins s'ils ont une frontière commune.

1 Compréhension du schéma (6 pts)

1. Donner les commandes de création de table pour `Département` et `Voisins`. On supposera que `Personne` et `Région` ont déjà été créées.
2. On ajoute la commande suivante au schéma de la table `Voisins`

   ```
   check (codeDpt1 < codeDpt2)
   ```

   Quelle est à votre avis la motivation de cette contrainte ?
3. Donner le schéma entité-association correspondant à ce schéma relationnel.
4. Existe-t-il une association plusieurs-plusieurs dans ce schéma ? Si oui vous paraîtrait-il judicieux de la réifier ? Justifiez votre réponse.
5. Une même personne peut-elle présider plusieurs départements ? Peut-elle présider un département et une région ?
6. Notons `D` le département, `P` la personne, `ND` le nom du département et `NP` le nom de la personne. Le schéma nout donne les dépendances fonctionnelles suivantes : `D -> P, ND` et `P -> NP`.

   On ajoute la dépendance `P -> D`. Comment l'interpréter et que devient le schéma relationnel ?

Dans toute la suite de l'examen, on travaille sur le schéma donné dans l'énoncé, et en supposant que la contrainte sur l'ordre des codes de département est respectée dans la table `Voisins` (donc, `codeDpt1 < codeDpt2`).

Correction

1. Standard. Voici pour la table `Voisins` :

```sql
create table Voisins (idDpt1 int not null,
    idDpt2 int not null
     primary key (idDpt1, idDpt2),
     foreign key (idDpt1) references Département (idDpt),
         foreign key (idDpt2) references Département (idDpt)
     )
```

2. Il existe une symétrie dans la notion de voisin : si A est voisin avec B, B est voisin avec A. Pour éviter le doublon de représentation (voisin(A, B) et voisin(B,A)), on décide de n'accepter que le premier, d'où la clause `check`
3.
4. `Voisins` provient d'une association plusieurs-plusieurs. La contrainte sur la clé implique qu'un département ne peut pas être deux fois le voisin d'un autre. C'est ici tout à fait raisonnable, donc la réification est inutile.

2 Algèbre et vues (4 pts)

On définit algébriquement les relations suivantes :
— $V1 := Voisins$
— $V2 := \rho_{codeDpt1 \to codeDpt2, codeDpt2 \to codeDpt1}(Voisins)$
— $V3 := V1 \cup V2$

1. Quel est le résultat de $V1 \cap V2$?
2. Comment caractériseriez-vous le contenu de $V3$?
3. Donnez la commande SQL de création de la vue $V3$.
4. Soit le département d_1 de code c_1. Quelle requête algébrique sur $V3$ donne les codes de *tous* ses voisins (expliquer).

Correction

1. Le résultat est vide !
2. Dans $V2$ on inverse la relation entre les codes de département. Donc, dans V2, `codeDpt2 < codeDpt1`). Du coup, dans V3, chaque relation de voisinage entre deux départements est représentée dans les deux sens.
3. On peut donc créer la vue suivante qui pourra nous simplifier les requêtes SQL par la suite.

```sql
create view V3 as
select *
from Voisins
union
select codeDpt2 as codeDpt1, codeDpt1 as codeDpt2
from Voisins
```

4. Avec V3, on n'a plus besoin de se soucier de l'ordre des codes de département. On peut donc écrire soit :

$$\pi_{codeDpt2}(\sigma_{codeDpt1=c_1}(\text{V3}))$$

ou

$$\pi_{codeDpt1}(\sigma_{codeDpt2=c_1}(\text{V3}))$$

qui donneront le même résultat.

3 SQL (8 pts)

Exprimez en SQL les requêtes suivantes :

1. Qui préside (prénom, nom) la région contenant le département 'Cantal' ?
2. Quelles villes sont à la fois préfecture de région et d'un département de plus de 100 000 habitants (donner le nom de la ville, du département, et l'intitulé de la région).
3. Quels départements d'une même région sont voisins l'un de l'autre ? Donnez l'intitulé de la région et les noms des deux départements. Attention à bien prendre en compte la contrainte sur les clés dans `Voisins`
4. Donner les paires de régions ayant une frontière commune (nom des régions).
   Aide : et si vous utilisiez la vue V3 définie précédemment ?
5. Quelles régions n'ont pas de département ?
6. Quelles personnes ne président ni région ni département ?
7. Donner, pour chaque région, le nombre de département et sa population totale (obtenue par cumul de celle des départements)
8. Dans la requête précédente, que se passe-t-il si une région n'a pas de département ? Comment réussir à afficher le nom de la région avec la valeur 0 pour le nombre de départements dans ce cas ?

Correction

1. ```
   select prénom, nom
   from Personne as p, Région as r, Département as d
   where d.nom='Cantal'
   and d.codeRégion = r.codeRégion
   and r.idPrésident = p.idPersonne
   ```

2. ```
   select r.préfecture as ville, r.intitulé, d.noms
   from  Région as r, Département as d
   where r.préfecture = d.préfecture
   and d.population > 10000
   ```

3.
```sql
select r.intitulé, d1.nom as nomDpt1, d2.nom as nomDpt2, Voisins as v
from  Région as r, Département as d1, Département as d2
where r.codeRégion = d1.codeRégion
and r.codeRégion = d2.codeRégion
/* Pour prendre la relation 'voisins dans le bon sens' */
and d1.codeDpt < d2.codeDpt
and v.codeDpt1=d1.codeDpt
and v.codeDpt2=d2.codeDpt
```

4. Solution avec la vue V3.

```sql
select r.intitulé, d1.nom as nomDpt1, d2.nom as nomDpt2
from  Région as r1, Région as r2, Département as d1, Département as d2, V3 as v
where d1.codeRégion = r1.codeRégion
and d2.codeRégion = r2.codeRégion
/* On sait que dans V3, on trouve chaque voisinage représenté dans les deux sens */
and v.codeDpt1=d1.codeDpt
and v.codeDpt2=d2.codeDpt
and r1.codeRégion != r2.codeRégion
```

Sinon, solution SQL complète avec un 'ou' logique

```sql
select r1.intitulé as nomRégion1, r2.intitulé as nomRégion2
from  Région as r1, Région as r2, Département as d1,
          Département as d2, Voisins as v
where d1.codeRégion = r1.codeRégion
and d2.codeRégion = r2.codeRégion
and (
      (d1.codeDpt < d2.codeDpt and v.codeDpt1 = d1.codeDpt1 and v.codeDpt2=d2.codeDpt)
    or
      (d1.codeDpt > d2.codeDpt and v.codeDpt1 = d1.codeDpt2 and v.codeDpt2=d1.codeDpt)
     )
```

Seconde solution avec une union

```sql
select r1.intitulé as nomRégion1, r2.intitulé as nomRégion2
from  Région as r1, Région as r2,
        Département as d1, Département as d2, Voisins as v
where d1.codeRégion = r1.codeRégion
and d2.codeRégion = r2.codeRégion
and d1.codeDpt < d2.codeDpt
and v.codeDpt1 = d1.codeDpt1
and v.codeDpt2 = d2.codeDpt
union
select r1.intitulé as nomRégion1, r2.intitulé as nomRégion2
from  Région as r1, Région as r2,
        Département as d1, Département as d2, Voisins as v
where d1.codeRégion = r1.codeRégion
and d2.codeRégion = r2.codeRégion
and d1.codeDpt > d2.codeDpt
```

```
and v.codeDpt1 = d2.codeDpt1
and v.codeDpt2 = d1.codeDpt
```

5.
```
select *
from  Région as r
where not exists (select* from Département as d
                  where r.codeRégion = d1.codeRégion)
```

6.
```
select *
from  Personne as p
where not exists (select* from Département as d
                  where d.idPrésident = p.idPrésident)
and not exists (select* from Région as d
                  where r.idPrésident = p.idPrésident)
```

7.
```
select r.intitulé, count(d.codeDpt), ssum(population) as population
from  Région as r, Département as d
where r.codeRégion = d1.codeRégion
group by r.codeRégion
```

8. Si on veut prendre en compte les régions sans département, il faut faire une jointure externe

```
select r.intitulé, count(d.codeDpt), sum(population) as population
from  Région as r left join Département as d on r.codeRégion = d1.codeRégion
group by r.codeRégion
```

Les régions sans département apparaissent dans le résultat de la jointure avec une valeur à `null` pour la population, et le `count` appliqué à `null` renvoie `0` (tandis que le `sum` appliqué à `null` renvoie `null`).

4 Un peu de logique (2 pts + 2 pts bonus pour les deux dernières questions)

Supposons les prédicats suivants :
— Competence(p, c) indique que la personne p a la compétence c
— OffreEmploi(o, c) indique que l'organisation o offre un emploi pour la compétence c
— FaitLaffaire(p, o) indique que la personne p fait l'affaire pour un emploi dans l'organisation o.
On suppose que la règle r suivante est toujours vraie :

$$\text{OffreEmploi}(o, c) \wedge \text{Competence}(p, c) \rightarrow \text{FaitLaffaire}(p, o)$$

Voici une petite mise en application dees règles de raisonnement que nous avons abordées. Si vous n'arrivez pas à exprimer une démonstration mathématique, essayez au moins de formuler des réponses intuitives (mais claires).

1. Reformulez la règle r sans implication (rappel : $p- > q \equiv \neg p \vee q$)
2. Dans la figure ci-dessous, les Dupondt peuvent-ils considérer de manière certaine que le capitaine Haddock les traite de clowns ? Formulez la phrase du capitaine selon les prédicats logiques ci-dessus, et vérifiez si on peut en déduire que les Dupondt ont la compétence 'clown'.

3. À tort ou a raison, les Dupondt se sentent offensés. Le capitaine Haddock propose une reformulation (figure ci-dessous).

Cette reformulation revient-elle vraiment à dire que les Dupondt *ne sont pas* des clowns ?

4. Quelle serait la bonne formulation d'une phrase qui indiquerait explicitement que le capitaine ne considère pas les Dupondt comme des clowns ?

Correction

— $p \to q$ est équivalent à $\neg p \vee q$. Donc r est équivalent à

$$r \equiv \neg\text{OffreEmploi}(\text{o},\text{c}) \vee \neg\text{Competence}(\text{p},\text{c}) \vee \text{FaitLaffaire}(\text{p},\text{o})$$

— Le capitaine énonce la conjonction de deux faits :

$$\text{OffreEmploi}('\text{Hipparque}','\text{clown}') \wedge \text{FaitLaffaire}('\text{Dupondt}','\text{Hipparque}')$$

Si $\text{FaitLaffaire}('\text{Dupondt}','\text{Hipparque}')$ est vrai, alors $\neg\text{Competence}(\text{p},\text{c})$ peut être vrai ou faux sans remettre en question la vérité de la règle r. Rien dans notre modèle logique ne permet donc d'en déduire que les Dupondt sont des clowns ! Il faudrait pour cela que notre règle r soit une *équivalence*, pas une *implication*.

— Cette fois le capitaine énonce les faits

$$\neg\text{OffreEmploi}('\text{Hipparque}','\text{clown}') \wedge \neg\text{FaitLaffaire}('\text{Dupondt}','\text{Hipparque}')$$

Ici encore, $\neg\text{Competence}(p, c)$ peut être vrai ou faux sans remettre en question la vérité de la règle r. Les Dupondt pourraient être des clowns !

— Regardons notre règle formulée sans implication. Pour que $\neg\text{Competence}('\text{Dupondt}', '\text{clown}')$ soit vrai sans ambiguité (donc les Dupondt ne sont pas des clowns), il faut que $\neg\text{OffreEmploi}('\text{Hipparque}', '\text{clown}')$ soit faux et $\text{FaitLaffaire}('\text{Dupondt}', '\text{Hipparque}')$ soit faux également.

$$\text{OffreEmploi}('\text{Hipparque}', '\text{clown}') \wedge \neg\text{FaitLaffaire}('\text{Dupondt}', '\text{Hipparque}')$$

Le capitaine devrait dire : “Le cirque Hipparque a besoin de deux clowns, mais vous ne faites pas l’affaire”. CQFD.